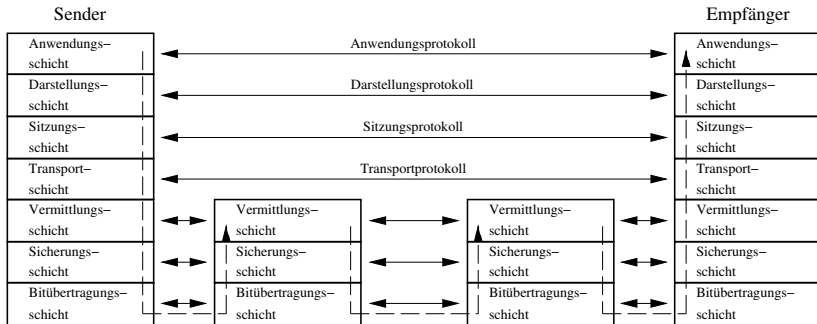


10. Transportprotokolle UDP und TCP

Das Transportprotokoll ist ein **Ende-zu-Ende-Protokoll** zwischen Quell- und Zielrechner.



Internet Protocol Stack

Schicht 5-7	ssh	SMTP	Telnet	FTP	HTTP	NFS/ DNS,	SNMP	NTP
4	TCP						UDP	
3	IP							
1-2	Schicht 1 + 2							

Beispiele: TCP (Transmission Control Protocol),
UDP (User Datagram Protocol)

Wesentliche Aufgabe der Transportschicht ist die Erhöhung der Dienstqualität des Vermittlungsdienstes:

- ggf. Paketverlust überwachen und korrigieren,
- ggf. Auf- und Abbau von Transportverbindungen
- Flußkontrolle zwischen Quell- und Zielrechner
- ggf. Aufteilen der Nachricht in kleinere Dateneinheiten für Schicht 3
- Multiplexing oder Splitting auf Verbindungen der Schicht 3

10.1 UDP (USER Datagram Protocol)

- Jon Postel, RFC 768, 1980
- Benutzerschnittstelle zum IP
- verbindungsloses Transportprotokoll (Schicht 4),
- keine Fehlerkontrolle $\implies$ Pakete können verloren gehen, dupliziert werden
- Postfach-Adressierung an sogenannte **Ports**

Woher kennt der Client den Server-Port?

1. Well-Known Port Assignment für Standardanwendungen.

In `/etc/services` sind die **Well Known Port Numbers** aufgelistet, z.B.:

```
# WELL KNOWN PORT NUMBERS
```

```
#  
ftp-data      20/tcp      #File Transfer [Default Data]  
ftp           21/tcp      #File Transfer [Control]  
telnet        23/tcp  
smtp          25/tcp      mail #Simple Mail Transfer  
http          80/tcp      www www-http #World Wide Web HTTP  
snmp          161/udp     #Simple Network Management Protocol
```

2. Dynamic Binding: Beim Erzeugen des Sockets wird dem Socket dynamisch vom Betriebssystem der nächste freie Port zugewiesen.

- ① Sinnvoll auf Client-Seite!
- ② Aber: **Woher kennt der Client den Server-Port?**

⇒ **Auskunftsdienst**

Der Server meldet sich mit seiner Portnummer bei einem Auskunftsdienst an.

Der Auskunftsdienst wiederum muß eine Well-Known Port Number besitzen.

Beispiel: Sun *RPC* benutzt **portmapper**: $\text{programm name} \mapsto \text{port}$
Bei Java RMI heißt der Auskunftsdienst **RMIRegistry**.

Socket-Adressen

- Adressen werden in einer allgemeinen Struktur angegeben, die aus einem Feld für die Protokollfamilie, einer Portnummer und einer für diese Protokollfamilie spezifischen Adresse besteht:

```
struct sockaddr_in {  
    sa_family_t    sin_family; /* address family: AF_INET */  
    u_int16_t      sin_port;   /* port in network byte order */  
    struct in_addr  sin_addr;   /* AF_INET: internet address */  
};
```

Wie erfährt der Server den Port des Clients?

Pakete heißen **User Datagram**.

Paketformat:

0	16	31
source port	destination port	
message length	checksum	
data		
...		

Pakete heißen **User Datagram**.

Paketformat:

0	16	31
source port	destination port	
message length	checksum	
data		
...		

source port: Verwendung ist optional, falls Antworten geschickt werden sollen, sonst source port = 0...0

Pakete heißen **User Datagram**.

Paketformat:

0	16	31
source port	destination port	
message length	checksum	
data		
...		

Paketlänge (Header + Daten):

8 Byte $\leq$ message length $\leq 2^{16} - 1 = 65\,535$ Byte

$\Rightarrow$ maximale UDP Datagramgröße!

Pakete heißen **User Datagram**.

Paketformat:

0	16	31
source port	destination port	
message length	checksum	
data		
...		

Verwendung der Prüfsumme (checksum) ist optional:
 $\text{checksum} = 0 \dots 0 \hat{=}$ Prüfsumme wird nicht benutzt

Prüfsumme gemäß Einerkomplement wird berechnet für:

UDP-Header mit checksum = 0	Pseudoheader	user data	Padding
-----------------------------	--------------	-----------	---------

Padding: auf Vielfaches von 16 Bit mit Nullen auffüllen

UDP-Pseudoheader:

0	8	16	31
SOURCE IP ADDRESS			
DESTINATION IP ADDRESS			
00000000	PROTOCOL NUMBER	UDP LENGTH	